Project 5: Monte Carlo average HashTable

Objective

We want to verify that the average case of inserting n elements into a HashTable with D buckets is approximately equal to n/4D. To get the average case of inserting n elements, we accomplished this project by the approach of Monte Carlo.

Monte Carlo

Estimated result = Random independent sampling + Many repetitions

In this project, random independent samples were generated by a random sequence of n integers, where each integer is range from 1 to 10000. The maximum input size n is 1000 in this project. Therefore, it is unlikely to have duplicate elements with the same key and ensure keys to be mostly independent. Many repetitions were accomplished by repeating inserting n elements 1000 times and finally took the Monte Carlo Average by Total comparisons / 1000.

Source Code

First is the foundation of the hash table, which is a sorted linked list. I copied it from the course webpage and modified the insert method so it can calculate the number of comparisons when insert an element.

```
#pragma once
#include <iostream>

using namespace std;

//exception is thrown if wrong input
class BadInput {
public:
    BadInput() {};
};
```

```
template <class E, class K>
struct Node {
    E data;
    Node<E, K>* next;
};

template <class E, class K>
class SortedChain {
public:
    SortedChain() { first = 0; }
    ~SortedChain();
    bool IsEmpty() { return (first == 0); }
    //int Length() const;
    //bool Search (const K& k, const E& e);
    SortedChain<E, K>& Delete(const K& k, E& e);
    int Insert(const K& k, const E& e);
    void Output() const;
private:
    Node<E, K>* first;
};


class TypeE {
public:
    long key;
    long value;
    TypeE() {};
    long Key() const { return key; }
    long Value() const { return value; }
};

// implementations

template <class E, class K>
SortedChain<E, K>::~SortedChain()
{
    Node<E, K>* current = first;
    while (first)
    {
        current = first->next;
```

```
        delete first;
        first = current;
    }
}
template <class E, class K>
SortedChain<E, K>& SortedChain<E, K>::Delete(const K& k, E& e)
{
    Node<E, K>* prior = first, * current = first;
    while (current && current->data.Key() < k) {  //search until
key=k or key > k
        prior = current;
        current = current->next;
    }
    if (current && current->data.Key() == k) {  //
        e = current->data;
        if (first == current) //delete first node
            first = current->next;
        else
            prior->next = current->next;
        delete current;
        return *this;
    }
    else
        throw BadInput();
}

template <class E, class K>
int SortedChain<E, K>::Insert(const K& k, const E& e)
{
    Node<E, K>* prior = first, * current = first;

    // the number of comparisons
    int comparisons = 0;

    while (current && (current->data).Key() < k) {
        // current data's key < k
        comparisons++;
        prior = current;
        current = current->next;
    }
```

```
    if (current && (current->data).Key() == k) {
        // current data's key = k
        comparisons++;
        throw BadInput(); //another node with same key
    }
    else {
        // only count if current exists, current data's key > k
        if (current)
            comparisons++;

        Node<E, K>* newp;
        newp = new Node<E, K>;
        if (first == current) { //we like to insert on first
position
            first = newp;
        }
        else
            prior->next = newp;
        newp->next = current;
        newp->data = e;
    }
    return comparisons;

}

template <class E, class K>
void SortedChain<E, K>::Output() const
{
    Node <E, K>* current = first;
    //cout << endl;
    while (current) {
        cout << "(" << current->data.Key() << ", " << current-
>data.Value() << ") -> ";
        current = current->next;
    }
}
```

Next is the HashTable class. I also copied the source code from the course webpage and modified the insert method so it can calculate the number of comparisons where there is an element trying to insert into the hash table.

```
#pragma once
#include <iostream>
#include "dictio.h" // This is the header file containing the
definition of the class SortedChain (Dictionary) that we covered
previously
// See the source file for the dictionary that is in the web-page
for CSC727.
template <class E, class K>
class HashTable {
public:
    HashTable(int divisor = 11) { D = divisor;ht = new
SortedChain<E, K>[D]; }
    ~HashTable() { delete[] ht; }
    //bool Search (const K& k, const E& e);
    HashTable<E, K>& Delete(const K& k, E& e) { ht[k % D].Delete(k,
e);return *this; }
    int Insert(const K& k, const E& e) { return ht[k % D].Insert(k,
e);}
    void Output() const;
private:
    int D;
    SortedChain<E, K>* ht;
};


// Output
template <class E, class K>
void HashTable<E, K>::Output() const
{
    for (int i = 0; i < D; i++)
    {
        std::cout << "ht[" << i << "] : ";
        ht[i].Output();
        std::cout << std::endl;
    }
}
```

Output method was also rewritten to clearly display each sorted linked list in the hash table line by line, which is only for testing the functionality of hash table.

Next is the function for calculating the Monte Carlo Average based on the given n (the number of elements needed to insert), D (the number of buckets) and trials.

```
long long MonteCarloAverage(int n, int D, int trials)
{
    // the total number of comparisons
    long long totalComparisons = 0;

    for (int step = 0; step < trials; step++)

    {   // new empty table each trial
        HashTable<TypeE, long> ht(D);

        // current trial comparisons
        long long curTrialComparisons = 0;

        for (int i = 0; i < n; i++)
        {
            TypeE x;
            // each key has a range from 1 to 10000
            x.key = rand() % 10000 + 1;
            x.value = x.key;

            try
            {
                curTrialComparisons += ht.Insert(x.key, x);
            }
            catch (BadInput)
            {
                // ignore duplicate keys
            }
        }

        totalComparisons += curTrialComparisons;
    }

    return totalComparisons / trials;
```

```
}
```

The calculated average is by n/4D

```
double CalculatedAverage(int n, int D)
{
    return static_cast<double>(n) / (4.0 * D);
}
```

Finally, the main function is to call out MonteCarloAverage and CalculatedAverage by fixing D=50, trials = 1000 and three different n: 100, 500 and 1000.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <iomanip>
#include "hashtable.h"
#include <vector>
using namespace std;
int main()
{
    srand((unsigned)time(0));

    const int D = 50;
    const int trials = 1000;

    vector<int> sizes = { 100, 500, 1000 };

    cout << left << setw(20) << "Input Size(n)" << setw(10) <<
"D=50" << setw(25) << "Calculated Average" << setw(25) << "Monte
Carlo Average" << endl;

    for (int i = 0; i < sizes.size(); i++)
    {
        int n = sizes[i];
        double calcAvg = CalculatedAverage(n, D);
        long long mcAvg = MonteCarloAverage(n, D, trials);
```

```
        cout << left << setw(20) << n << setw(10) << "" << setw(25)
<< fixed << setprecision(2) << calcAvg << setw(25) << mcAvg <<
endl;
    }

    return 0;
}
```

Output

```
Input Size(n)       D=50      Calculated Average       Monte Carlo Average
100                           0.50                     82
500                           2.50                     1526
1000                          5.00                     5273
```

Since Calculated Average is by n/4D, which represents the average number of comparisons per insertion. However, Monte Carlo Average computes the total comparisons per trial. To make them comparable, the Monte Carlo Average should be divided by the number of insertions n.

The updated Output by letting Monte Carlo Average divided by n:

```
Input Size(n)       D=50      Calculated Average       Monte Carlo Average
100                           0.50                     0.83
500                           2.50                     3.05
1000                          5.00                     5.28
```

Error Rate

n = 100: Error = |0.50 - 0.83| / 0.50 = 66%

n = 500: Error = |2.50 - 3.05| / 2.50 = 22%

n = 100: Error = | 5.00 - 5.28| / 5.00 = 5.6%

Conclusion

The error rate become smaller as n become larger. Therefore, we verified the average number of comparisons when inserting n elements into D buckets is n/4D.